

FINLEDGER · ACCOUNTS PAYABLE AUTOMATION

Person 1 — AI / RAG / Tokens

FinLedger AP · India · Tally-first · September 2026 Seat: extract LLM, risk-LLM slice, client-scoped RAG, VoucherDraft, prompts, token budget, correction memory.

Product (read this first)

FinLedger is a per-client Accounts Payable machine for Indian CAs and SMBs. Vendors send bills to one official link or one official email. The platform extracts the invoice, scores risk, maps ledgers with client-scoped RAG, waits for a human when the bill is risky, then posts an approved VoucherDraft into that client's ERP. Tally Prime first.

What it does. Takes unstructured vendor PDFs/photos and turns them into posted purchase vouchers with open bills that age. GST / TDS / MSME native. Not a GST filer, not a bank, not a card issuer.

How a bill moves. Vendor sends → Person 2 stores markdown/images → Person 1 extracts CanonicalInvoice and maps a VoucherDraft via RAG → Person 3 scores and reviews → Person 4 posts to Tally. Extraction never posts. The mapping LLM never emits Tally XML.

Full story. docs/PRODUCT.md and docs/HOW_IT_WORKS.md.

Mission

Turn a stored document into a structured CanonicalInvoice, a RiskScore LLM-slice, and an ERP-agnostic VoucherDraft. Never post. Never emit Tally XML / Zoho JSON / SAP OData. Never mix clients in one index.

One-line path Person 1 owns:

markdown + page images → CanonicalInvoice → LLM risk marks → VoucherDraft + reasons + MapTrace

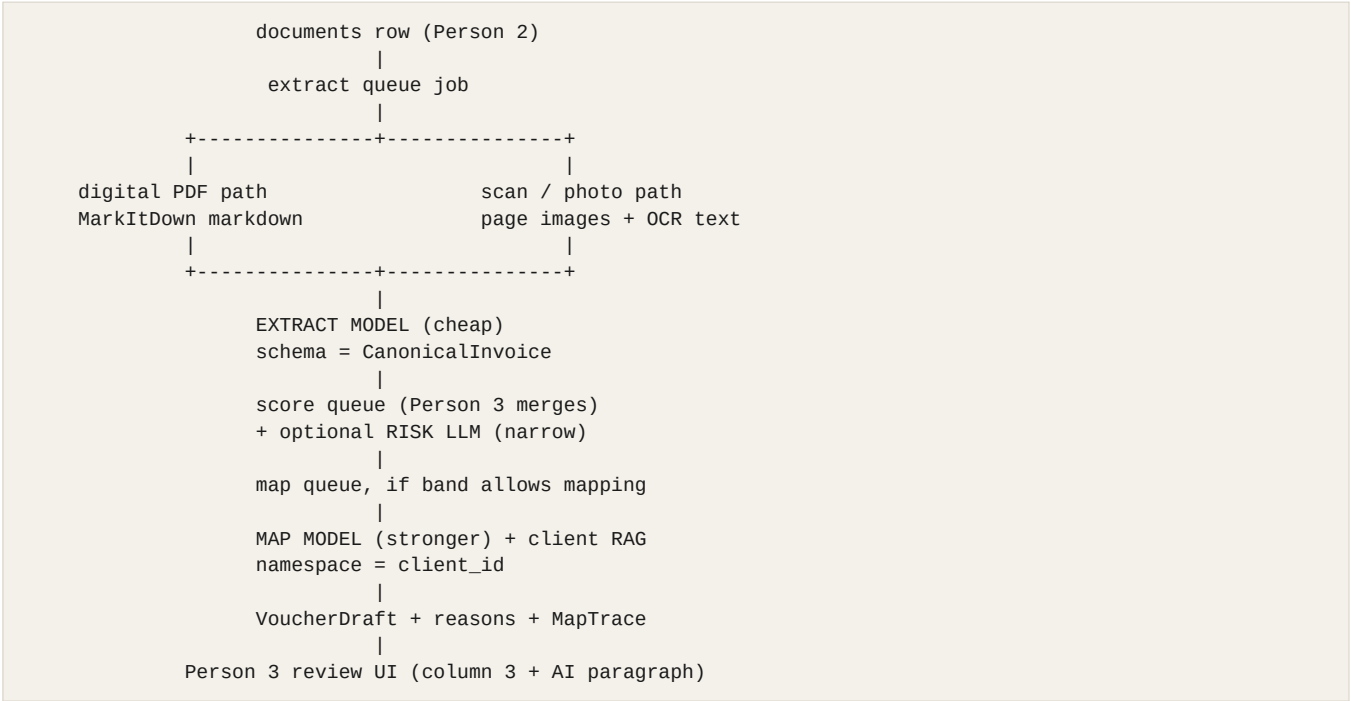
Layers owned

From the seven-layer architecture:

#	Layer	Person 1 piece
3	Extract	Schema-constrained LLM on Markdown output. Vision only for scans.
4	Control	LLM fraud / language marks only. Code checks belong to Person 3.
5	AP brain	Vendor resolve assist, RAG + hard rules → VoucherDraft. GST/TDS suggestions.
12	Review UI	AI approval paragraph only. Not the UI chrome.

Not owned: channels, object store, state machine transitions, review screen layout, Tally agent, payment rails.

Runtime architecture



Two calls, not one mega-prompt. Extract never sees ERP ledgers. Map never sees raw pixels first.

RAG design

- Namespace / collection prefix = client_id. Hard isolation.
- Do not build one global ledger brain.
- Retrieval query = vendor GSTIN + name + line descriptions + HSN + amount band + document_type.

Index these chunk types per client:

type	source	who writes
ledger	chart of accounts / Tally groups and ledgers	Person 4 masters → Person 2 store → Person 1 embed
vendor	vendor master (GSTIN key)	same
posted_bill	last N posted invoices for that vendor	Person 4 post success → Person 1 embed
memory	CorrectionEvent (this vendor + HSN + desc → ledger + tax)	Person 3 edit → Person 1 upsert
policy	PO mandatory above X, rent → 194I, auto-post cap	Person 3 Client policy
hard_rule	library of GST/TDS/MSME rules, not client-specific but applied per client	Person 1

Output of map: party ledger (create-proposal if missing), purchase/expense split, GST input ledgers, TDS if policy hits, bill-wise New Ref = invoice_no, due date = invoice_date + credit days, narration, reasons. Store retrieved chunks on MapTrace.

Token / model architecture

Call	Model class	Input	Cap
extract	cheap / fast	truncated markdown + optional table blocks	hard token cap per bill
extract_vision	multimodal	selected page images + OCR text	only if digital text empty
risk_llm	small	invoice fields + sender meta + prior vendor	optional, skip if extract conf high and identity clean
map	stronger	CanonicalInvoice + retrieved chunks + policy	cap chunks (e.g. top 8)
summary	cheap	draft + risk band + vendor history	4–6 lines

Rules:

- Log every call: client_id, doc_id, model, input/output tokens, latency, cost, purpose.
- Per-bill budget + kill switch. Fat PDF cannot blow the month.
- Prompt cache where the provider allows.
- If map model is down: leave bill in extracted / scored. Never post.
- LLM never writes connector payloads.

Workers Person 1 ships

1. extract_worker — input document_id; output CanonicalInvoice + confidences. 2. risk_llm_worker — optional marks merged by Person 3. 3. map_worker — input invoice + client_id; output VoucherDraft + MapTrace. 4. memory_indexer — consumes CorrectionEvent and posted bills. 5. usage_writer — token/cost rows. 6. eval_harness — fixture pack of 30–50 Indian bills.

Interfaces

From Person 2

- document_id
- raw_markdown path
- layout_blocks (page, bbox, table cells)
- page_image_paths
- client_id, source channel, hash

To Person 3

- CanonicalInvoice
- LLM risk marks
- VoucherDraft
- AI approval paragraph
- MapTrace id

To Person 4

- Nothing except VoucherDraft. If Tally needs a field, add it to the draft schema.

From Person 3

- CorrectionEvent on every reviewer edit
- Client policy flags the mapper must read, not invent

From Person 4

- Masters (ledgers, vendors) after sync, so RAG can index
- Posted bill ids after successful post

Week plan (Person 1)

Week	Ship
0	Schemas, model router, usage table, 20-bill fixture set
1	Digital PDF extractor → CanonicalInvoice. Cost dashboard.
2	Client RAG + mapper → VoucherDraft. Propose vendor, never auto-create.
3	Scan/vision path. AI approve paragraph. Eval harness.
4	Correction memory live. Tune prompts from real edits.

Acceptance tests

- Two clients’ embeddings cannot retrieve each other.
- Extract on a digital GST invoice returns invoice_no, GSTIN, totals, at least one HSN line.
- Line sum ≠ header → mark, do not rewrite total.
- Mapper output validates against VoucherDraft schema. No XML tags.

- New vendor sets `party_create_proposal=true`.
- CorrectionEvent for “HSN 9983 → Professional fees 194J” is used on the next bill from that vendor.
- Usage row exists for every model call.
- Kill switch fires when a bill exceeds the token budget.

Refusals for this seat

- Global RAG
- Raw pixels into the mapping prompt first
- LLM-written Tally XML
- Skipping review because the model is confident
- Auto-create vendor / ledger
- Inventing tax totals to force balance